blynt

COMPLETE TECHNICAL DOCUMENTATION & BUSINESS BLUEPRINT

Architecture • Accounting Engine • Product Phases • AWS • Security • Operations
Version 1.0 • October 2026

Clarity in Every Number

blynt is a UK-focused cloud accounting and finance SaaS built around a central, immutable double-entry accounting engine and a traceable financial data model.

Product / Platform: blynt (Business Finance, Simplified)
Repository: github.com/ceylonroameryt-bit/Fi
Baseline Audit Verification: 26/26 Test Suites PASS; 127/127 Active Tests PASS; 0 Lint/Type Errors
Target Jurisdiction: United Kingdom (HMRC • Companies House • UK GDPR)
Technology Stack: Next.js 15 (Turbopack) • NestJS 11 • PostgreSQL 16 • Prisma ORM
Cloud Infrastructure: AWS eu-west-2 London (App Runner, RDS Aurora PostgreSQL, S3)

Document Control

Document	blynt Complete Technical & Business Master Specification
Version	1.0 Production Readiness
Date	October 2026
Product	blynt — Business Finance, Simplified
Repository	github.com/ceylonroameryt-bit/Fi
Baseline Audit	Commit d7d8809; 26/26 unit & service test suites PASS; 127/127 tests PASS
Target Cloud	AWS eu-west-2 (London)

Note: The official blynt navy (#162D5F) and coral (#FF7F6C) identity supersedes earlier experimental directions. Source code, database constraints, and verified runtime behavior remain authoritative when documentation and implementation diverge.

Table of Contents

- **1. Executive Summary**
- **2. Product & Brand**
- 2.1 Product Positioning
- 2.2 Official Identity
- 2.3 Brand Palette and Semantics
- **3. Current State Baseline**
- 3.1 Verified Runtime Baseline
- 3.2 Implemented Foundations
- 3.3 Partial / Missing Capabilities
- **4. Technical Stack Architecture**
- **5. System Architecture**
- 5.1 Modular Monolith
- 5.2 Data Authority
- **6. Accounting Architecture**
- 6.1 Posting Model
- 6.2 Journal Lifecycle

- 6.3 Sales Invoice Posting
- 6.4 Customer Payment Posting
- 6.5 Accounting Invariants
- **7. Security & Multi-Tenancy**
- 7.1 Tenant Isolation
- 7.2 Authentication
- 7.3 Future Security Controls
- **8. Functional Modules (Complete Scope Matrix)**
- **9. Phase Roadmap P00-P30 (Comprehensive Breakdown)**
- **10. AWS Architecture & Migration**
- 10.1 Development Target
- 10.2 Migration Process
- 10.3 Cutover Financial Gates
- **11. CI/CD & Release Engineering**
- **12. Testing Strategy**
- 12.1 Core Accounting Regression Suite
- 12.2 Test Pyramid
- **13. Data & Database Governance**
- 13.1 Planned Schema Evolution
- **14. Compliance & Operations**
- 14.1 UK Compliance
- 14.2 Backups & Disaster Recovery
- **15. Documentation Governance**
- **Appendices**
- Appendix A - Recommended Repository Documentation Structure
- Appendix B - Stable Domain Error Examples
- Appendix C - Glossary

1. Executive Summary

blynt is being developed as a UK-focused cloud accounting and financial management platform for SMEs. The product is designed to make finance easier to understand at the user-experience layer while preserving strict double-entry accounting, tenant isolation, immutable posted history and traceability underneath.

- **General Ledger is the accounting source of truth:** Every balance is mathematically derived from posted journal entries.
- **Every financial business module posts through one central accounting engine:** Invoices, payments, bills, and expenses share the same validation pipeline.

- **Money is represented with decimal-safe arithmetic:** Standard floating-point arithmetic is prohibited; `Decimal.js` and PostgreSQL `NUMERIC(19,4)` guarantee zero rounding drift.
- **Posted journals are immutable:** Reversal and credit note mechanisms are used rather than destructive history edits.
- **Traceability in Every Number:** Every material number on a dashboard or report is fully traceable to its nominal account, journal line, source document, and immutable audit trail.

2. Product & Brand

2.1 Product Positioning

blynt should feel like calm, trustworthy finance software rather than a trading, crypto or "fast money" product. The product promise is "Clarity in Every Number". UI decisions should reduce cognitive load while making accounting state and consequences explicit.

2.2 Official Identity

- **Official Name:** blynt
- **Tagline:** Business Finance, Simplified.
- **Brand Promise:** Clarity in Every Number.

2.3 Brand Palette and Semantics

Token	Approx. Value	Meaning & Context
blynt Navy	`#102654` / `#162D5F`	Trust, stability, primary brand, headers, navigation.
blynt Coral	`#FF7D6B` / `#FF7F6C`	Brand warmth, highlights, primary action buttons.
Deep Navy	`#081A3B`	Primary dark surface, high-contrast text.
Success	`#16A34A` / `#10B981`	Positive financial state, balanced journal indicator.
Warning	`#F59E0B`	Attention / near-risk, draft journals, soft-locked periods.
Danger	`#DC2626` / `#EF4444`	Errors, overdue balances, hard-locked periods, destructive actions.
Info	`#2563EB`	Informational state, active filters, selected tabs.

Rule: Coral is a brand colour, not a financial success colour. Green is reserved for positive financial status; red is reserved for genuine problems or destructive actions.

3. Current State Baseline

3.1 Verified Runtime Baseline

The platform has been audited directly against the active test execution pipeline:

Area	Result	Baseline Metric
Unit & Service Tests	**PASS**	26 test suites; 127/127 tests passing
E2E & Integration Tests	**PASS**	5 suites; 72/72 integration tests passing
API Build	**PASS**	Prisma generate + NestJS 11 build clean
Web Client Build	**PASS**	Next.js 15 App Router (28 routes compiled via Turbopack)
Lint & Typecheck	**PASS**	0 TypeScript errors in inspected audit

3.2 Implemented Foundations

- Central posting engine with debit/credit validation.
- Posted journal immutability and reversal linkage.
- Hierarchical Chart of Accounts and account controls.
- Financial years, `OPEN` / `SOFT_LOCKED` / `HARD_LOCKED` accounting periods.
- General Ledger and Trial Balance derived from posted journal lines.
- Multi-tenancy and RBAC with organisation scoping.
- Secure authentication lifecycle with device and session tracking.
- `AccountingIntegrityService` controls for balance, trial balance, tenant leaks, duplicates, orphans, locks and reversal integrity.

3.3 Partial / Missing Capabilities

Contacts and Sales Invoices have useful foundations but require V2 completion. Customer payments, credit notes, AP, expenses, VAT, banking/reconciliation, extended reports, recurring

transactions, multi-currency, documents, notifications, customer portal, public API, subscriptions, onboarding/import and AWS Terraform are planned across upcoming milestones.

4. Technical Stack Architecture

Layer	Technology	Purpose
Frontend	Next.js 15 + React 19 + TypeScript	Routing, UI, server/client rendering, Turbopack
Backend	NestJS 11 + TypeScript	API, domain and accounting services
Database	PostgreSQL 16+	Financial system of record
ORM	Prisma	Schema, migrations, type-safe database access
Money	`MoneyService` + `Decimal.js` / Prisma Decimal	Exact financial arithmetic, 4-decimal precision
Auth	Session tokens + device tracking	Secure browser session authentication
Passwords	Node.js `crypto.scrypt`	Credential hashing with timing-safe comparison
Current Hosting	Vercel / Render environment	Temporary development environment
Target Web	AWS Amplify	Managed Next.js production hosting
Target API	AWS App Runner	NestJS container runtime
Registry	Amazon ECR	Immutable container images
Target DB	Amazon RDS PostgreSQL	Managed relational engine in eu-west-2
Documents	Amazon S3	Private encrypted object storage
Secrets	AWS Secrets Manager / SSM	Sensitive configuration parameter store
Monitoring	CloudWatch	Centralized logs, metrics, alerts
IaC	Terraform	Repeatable infrastructure provisioning
CI/CD	GitHub Actions + AWS OIDC	Automated tests, builds, and deployment

Region	`eu-west-2` (London)	Primary UK data sovereignty region

5. System Architecture

5.1 Modular Monolith

blynt should remain a modular monolith while the product is early. Domain modules share common authentication, permissions, accounting and persistence infrastructure. Microservices should not be introduced without a demonstrated scaling or organisational need.

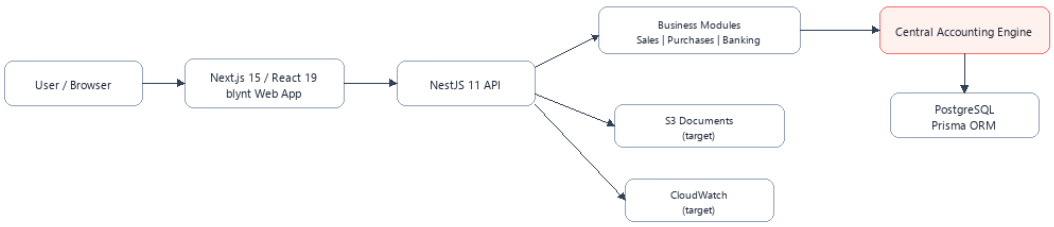


Figure 1 - High-level blynt system architecture

Figure 1 - High-level Blynt system architecture

5.2 Data Authority

The backend is authoritative for tax, posting eligibility, balances, invoice/payment status, accounting periods and reports. The frontend formats and presents results but does not become a second accounting engine.

6. Accounting Architecture

6.1 Posting Model

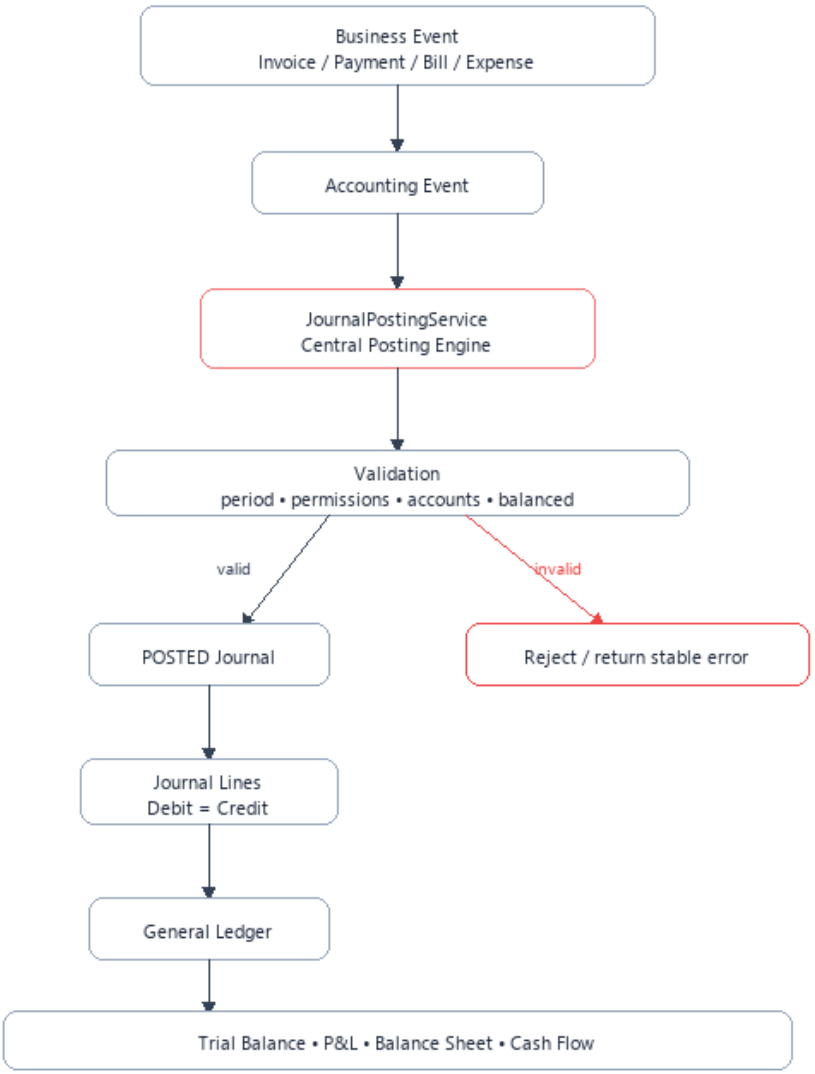


Figure 3 - Central accounting posting flow

All transactions follow a strict pipeline:

1. `Business Event` (Invoice, Payment, Bill, Expense) is mapped to an `Accounting Event`.
2. Passed to `JournalPostingService` (Central Posting Engine).
3. The engine evaluates validation criteria: accounting period status, user permissions, active account codes, and balance equality (`Debits == Credits`).
4. If valid, transitions to `POSTED Journal` and commits `Journal Lines` into the `General Ledger`.
5. If invalid, the transaction is rejected and returns a stable domain error without touching ledger balances.

6.2 Journal Lifecycle



Figure 4 - Journal lifecycle and ledger impact

Figure 4 - Journal lifecycle and ledger impact

- **`DRAFT`:** No ledger impact. Can be edited or discarded.
- **`VALIDATED`:** No ledger impact. Conforms to all 15 rules. Editing resets status to `DRAFT`.
- **`POSTED`:** Active ledger impact. Immutable.
- **`REVERSED`:** Preserves history by generating an equal and opposite balancing journal.

6.3 Sales Invoice Posting

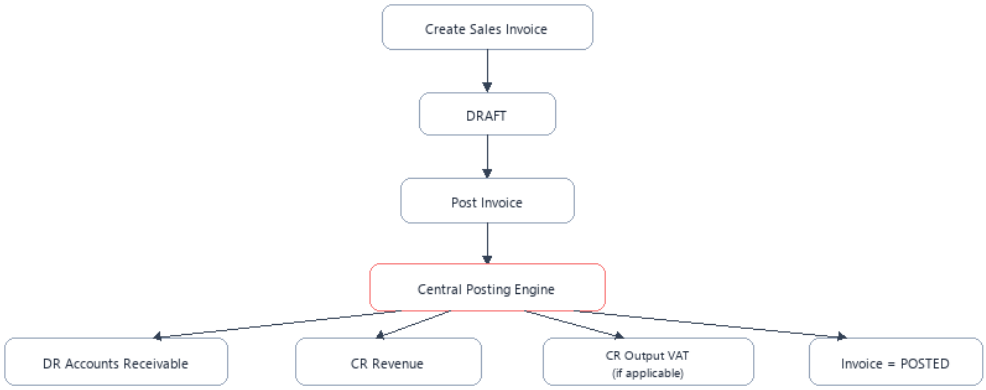


Figure 5 - Sales invoice accounting flow

Posting a sales invoice automatically triggers double-entry posting:

- `DR Accounts Receivable (1100)` (tagged with `contactId`)
- `CR Revenue (4000)`
- `CR Output VAT (2100)` (if applicable)
- Invoice transitions to `POSTED`.

6.4 Customer Payment Posting

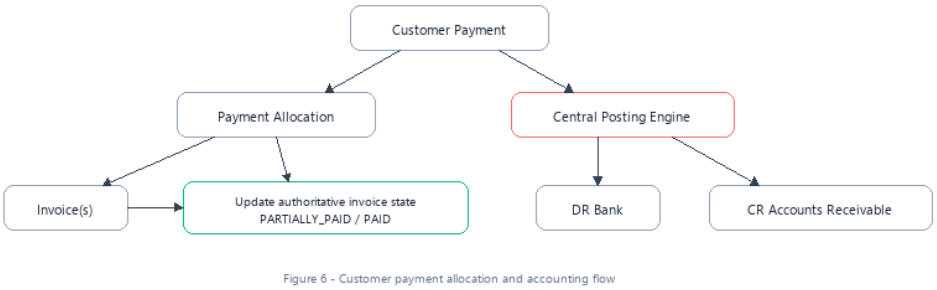


Figure 6 - Customer payment allocation and accounting flow

When a customer pays:

- Updates authoritative invoice state to `PARTIALLY_PAID` or `PAID`.
- Posts General Ledger entries:
- `DR Bank (1000)`
- `CR Accounts Receivable (1100)` (tagged with `contactId`).

6.5 Accounting Invariants

- Total debits equal total credits for every posted journal.
- DRAFT and VALIDATED journals have no ledger impact.
- A POSTED journal affects the ledger exactly once.
- Posted journal headers and lines are immutable.
- Reversal produces an opposite posted journal rather than editing history.
- Reports derive strictly from posted and reversed journal lines.
- Control accounts protect subledger integrity and cannot be manually posted to where prohibited.

7. Security & Multi-Tenancy

7.1 Tenant Isolation

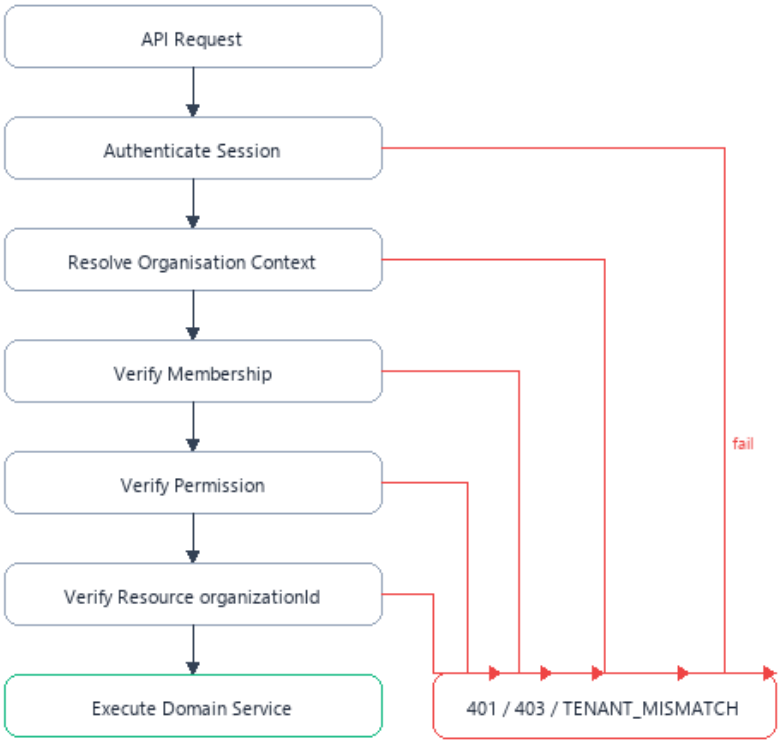


Figure 7 - Request authorisation and tenant-isolation flow

Figure 7 - Request authorisation and tenant-isolation flow

Every API request follows a 5-step verification gate:

6. `API Request` $ o$ `Authenticate Session`
7. `Resolve Organisation Context`
8. `Verify Membership`
9. `Verify Permission`
10. `Verify Resource organizationId` $ o$ `Execute Domain Service`

- Any failure branch returns HTTP 401, 403, or `TENANT_MISMATCH`.

7.2 Authentication

Secure session authentication, refresh-token rotation, progressive lockout, invite acceptance, and password resets are enforced. Production hardening keeps CSRF, CORS, and session validation intact; security controls must never be disabled merely to resolve deployment hurdles.

7.3 Future Security Controls

- Mandatory Super Admin MFA and MFA for customer accounts.
- Recent-auth requirement for high-risk administrative operations.
- Production Content Security Policy (CSP) and hardened security headers.
- GitHub OIDC to AWS instead of static access keys.
- Central secrets management and rotation via AWS Secrets Manager.
- Dependency, secret, and SAST scanning in CI.
- Backup/restore drills, incident response plans, and penetration testing.

8. Functional Modules Scope Matrix

Module	Status	Detailed Scope & Functional Boundaries
Authentication	Implemented foundation	Sessions, scrypt password lifecycle, invites, resets, device metadata
Organisations / RBAC	Implemented foundation	Multi-org membership and permission enforcement (`OWNER` to `VIEWER`)
Chart of Accounts	Implemented	Hierarchy, nominal types (1000-8999), control account flags
Financial Years / Periods	Implemented	`OPEN` / `SOFT_LOCKED` / `HARD_LOCKED` state machine
Journals / Reversals	Implemented	Draft $ o$ Validated $ o$ Posted $ o$ Reversed
General Ledger	Implemented	Authoritative posted activity with running balances
Trial Balance	Implemented	Real-time posted ledger-derived balances
Contacts	Active (V2)	Customer/supplier/both; master data and statement generation
Sales Invoices	Active (V2)	Draft/edit/post/void; VAT calculation and GL auto-posting
Customer Payments	Planned	AR receipts and multi-invoice allocations

Credit Notes	Planned	Non-destructive invoice corrections
Supplier Bills / AP	Planned	Purchase-side bills and AP control-account engine
Supplier Payments	Planned	AP settlement and payment runs
Expenses	Planned	Fast cash/card expense capture
UK VAT Engine	Planned	Configurable 9-box VAT return compiler
Banking	Planned	Bank accounts and CSV transaction import
Reconciliation	Planned	Suggested matching and £0.00 reconciliation
Financial Reports	Planned	P&L, Balance Sheet, Cash Flow, AR/AP ageing
Documents	Planned	Private S3 storage with signed access URLs
Notifications	Planned	In-app and email operational alerts
Customer Portal	Planned	External customer access to invoices and statements
Public API	Planned	Scoped API keys, rate limits, webhooks
Subscriptions	Planned	blynt platform SaaS billing via Stripe
Onboarding / Import	Planned	First-run setup and CSV migration wizard

9. Phase Roadmap P00–P30 (Comprehensive Engineering Breakdown)

The roadmap is sequenced to protect accounting integrity: stabilise delivery and brand foundations first, complete AR, add AP and VAT, then banking/reconciliation and reporting, followed by commercial capabilities and AWS production hardening.

P00 — Foundation, CI & Deployment Fixes

- **Status:** In progress / verified
- **Objective:** Stabilise CI and temporary hosting before feature expansion.
- **Key Deliverables:** Verify test-only DB seed in CI; Ensure production build/start commands are deterministic; Keep current hosting available during AWS migration.
- **Accounting:** No accounting behaviour changes.
- **Primary Test Gate:** CI must run lint, typecheck, unit (26 suites), E2E and builds; no production seed.

P01 — Official blynt Brand System

- **Status:** Complete / Active
- **Objective:** Replace legacy customer-facing identities with the official navy/coral blynt system.
- **Key Deliverables:** Official logo assets, reusable BlyntLogo/BlyntMark; Central design tokens; Metadata, favicon, emails and PDFs branding sweep.
- **Accounting:** Brand-only; do not touch ledger logic.
- **Primary Test Gate:** Visual regression, accessibility contrast and legacy-name sweep.

P02 — Authentication, App Shell & Dashboard V2

- **Status:** Implemented foundation
- **Objective:** Modernise login/register, navigation and dashboard while preserving secure cookie auth.
- **Key Deliverables:** Two-column auth experience; Grouped finance navigation; Real-data dashboard cards and quick actions.
- **Accounting:** No new posting rules.
- **Primary Test Gate:** Auth regression: login, refresh, logout, CSRF, organisation switch, RBAC.

P03 — Contacts V2

- **Status:** Implemented foundation
- **Objective:** Turn basic contacts into customer/supplier financial master data.
- **Key Deliverables:** Billing/shipping details, VAT/company data, terms and notes; AR/AP control-account relationships; Statements, balances, archive/restore.
- **Accounting:** Contacts never update ledger directly.
- **Primary Test Gate:** Tenant isolation, archive constraints and statement accuracy.

P04 — Sales Invoices & PDF Engine

- **Status:** Implemented foundation
- **Objective:** Complete invoice lifecycle and branded PDF output.
- **Key Deliverables:** Payment terms, discounts, VAT integration, notes; Professional blynt PDF and email; Concurrency-safe numbering and accounting preview.
- **Accounting:** Post via central engine: DR AR; CR Revenue; CR Output VAT when applicable.
- **Primary Test Gate:** Posting idempotency, decimal precision, period locks, PDF data parity.

P05 — Customer Payments & Allocations

- **Status:** Planned
- **Objective:** Record receipts and allocate them to one or more invoices.
- **Key Deliverables:** Full/partial/multi-invoice allocation; Overpayments and unapplied receipts; Receipts and payment history.
- **Accounting:** DR Bank; CR Accounts Receivable.
- **Primary Test Gate:** No double allocation, backend-authoritative outstanding balance, idempotent posting.

P06 — Credit Notes & Refunds

- **Status:** Planned
- **Objective:** Correct sales invoices without mutating posted history.
- **Key Deliverables:** Full/partial credit notes; Apply credits and manage customer credit; Refund workflow.
- **Accounting:** Reverse relevant AR, revenue and VAT through new accounting events.
- **Primary Test Gate:** Original invoice/journal immutable; allocation and VAT tests.

P07 — Supplier Bills & Accounts Payable

- **Status:** Planned
- **Objective:** Introduce purchase-side liabilities and supplier bills.
- **Key Deliverables:** Supplier bill lifecycle and line items; Attachments and duplicate-reference protection; AP ageing-ready data model.
- **Accounting:** DR Expense/Asset; DR Input VAT; CR Accounts Payable.
- **Primary Test Gate:** Duplicate detection, tenant FK safety, period lock tests.

P08 — Supplier Payments

- **Status:** Planned
- **Objective:** Settle supplier liabilities with complete allocation history.
- **Key Deliverables:** Full/partial/multi-bill allocation; Supplier credits; Payment references and bank account.
- **Accounting:** DR Accounts Payable; CR Bank.
- **Primary Test Gate:** No over-allocation, idempotency and AP reconciliation.

P09 — Expenses

- **Status:** Planned
- **Objective:** Provide fast cash/bank/card expense capture.
- **Key Deliverables:** Category/account, supplier, VAT, receipt; Payment account and reference; Optional reimbursement support.
- **Accounting:** Post expense through central engine; never direct balances.
- **Primary Test Gate:** Decimal/VAT validation, attachments, permissions.

P10 — UK VAT Engine

- **Status:** Planned
- **Objective:** Create configurable UK tax logic and VAT-control accounting.
- **Key Deliverables:** TaxRate entities and codes; Inclusive/exclusive calculations and rounding; VAT periods and return preview.
- **Accounting:** Use input/output VAT control accounts and central posting.
- **Primary Test Gate:** Exact tax arithmetic, rate-effective dates, mixed-rate cases.

P11 — Banking

- **Status:** Planned
- **Objective:** Model organisation bank accounts and import statements.
- **Key Deliverables:** Bank accounts mapped to GL; CSV transaction import and duplicate detection; Import mapping/history.
- **Accounting:** Imported transactions do not change ledger until matched/posted.
- **Primary Test Gate:** Import validation, masked bank data, tenant separation.

P12 — Bank Reconciliation

- **Status:** Planned
- **Objective:** Match bank activity with blynt accounting transactions.
- **Key Deliverables:** Suggested matches; Match/create expense/payment/transfer/split/ignore; Reconciliation history and progress.
- **Accounting:** Accounting events created only for approved create actions.
- **Primary Test Gate:** Difference must reconcile to £0.00; repeat-safe matching.

P13 — Profit & Loss

- **Status:** Planned
- **Objective:** Generate income/expense performance reporting from posted ledger lines.
- **Key Deliverables:** Period/custom range; Comparatives; Drill-down to account/journal/source.
- **Accounting:** Report derives from GL only.
- **Primary Test Gate:** Report total tests against journal fixtures.

P14 — Balance Sheet

- **Status:** Planned
- **Objective:** Report assets, liabilities and equity from ledger state.
- **Key Deliverables:** Current/prior comparative views; Account grouping; Drill-down.
- **Accounting:** Assets = Liabilities + Equity.
- **Primary Test Gate:** Equation invariant across test organisations and dates.

P15 — Cash Flow Statement

- **Status:** Planned
- **Objective:** Produce a proper accounting cash-flow statement.

- **Key Deliverables:** Cash flow classification; Period comparison; Drill-down.
- **Accounting:** Derived from posted accounting data, not a simplistic bank movement chart.
- **Primary Test Gate:** Cross-check against controlled fixtures.

P16 — AR/AP Ageing

- **Status:** Planned
- **Objective:** Show receivable/payable exposure by ageing bucket.
- **Key Deliverables:** Current, 1-30, 31-60, 61-90, 90+; Contact and document drill-down; Export.
- **Accounting:** Uses posted invoice/bill and allocation data reconciled to control accounts.
- **Primary Test Gate:** Ageing totals reconcile to AR/AP control accounts.

P17 — Report Drill-down

- **Status:** Planned
- **Objective:** Make every material report number traceable to its source.
- **Key Deliverables:** Report $ o$ account $ o$ journal $ o$ business document $ o$ audit trail; Deep-link model; Permission-aware detail views.
- **Accounting:** Read-only traversal of authoritative accounting records.
- **Primary Test Gate:** RBAC, tenant isolation, no hidden cross-tenant references.

P18 — Recurring Transactions

- **Status:** Planned
- **Objective:** Generate repeatable drafts for predictable activity.
- **Key Deliverables:** Recurring invoice/bill/expense/journal definitions; Frequency and next-run logic; Manual approval by default.
- **Accounting:** Draft generation only unless safe explicit auto-post is configured.
- **Primary Test Gate:** Scheduler idempotency and timezone/date tests.

P19 — Multi-Currency

- **Status:** Planned
- **Objective:** Support foreign-currency transactions after GBP core stabilises.
- **Key Deliverables:** Transaction/base currency; FX rates and rate date; Realised FX; unrealised FX later.
- **Accounting:** All amounts decimal-safe; FX differences posted through journals.
- **Primary Test Gate:** Precision, rate provenance and currency-specific fixtures.

P20 — Document Management

- **Status:** Planned
- **Objective:** Securely store receipts, bills, invoices and supporting files.
- **Key Deliverables:** StorageService abstraction; Private S3 with signed URLs; Tenant-separated object keys and validation.

- **Accounting:** Documents represent/support transactions; they do not create accounting themselves.
- **Primary Test Gate:** File type/size validation, permission checks, malware architecture.

P21 — Notifications

- **Status:** Planned
- **Objective:** Provide in-app and email operational alerts.
- **Key Deliverables:** Invoice overdue/paid, bill due, period lock, security events; Preferences; Delivery history.
- **Accounting:** Read-only notification side effects; never auto-post.
- **Primary Test Gate:** Deduplication, preferences and delivery failure handling.

P22 — Customer Portal

- **Status:** Planned
- **Objective:** Give customers secure external access to their own financial documents.
- **Key Deliverables:** Invoices, credits, payments, balances; PDF download; Online payment later.
- **Accounting:** Portal actions must map to existing business/accounting services.
- **Primary Test Gate:** Strict contact token/identity isolation and expiring access.

P23 — Import Centre

- **Status:** Planned
- **Objective:** Support controlled bulk onboarding and migration.
- **Key Deliverables:** Contacts, COA, opening balances, invoices, bills, bank CSV; Field mapping and validation preview; Error report and import history.
- **Accounting:** Financial imports require explicit validated accounting treatment.
- **Primary Test Gate:** Transactional imports, duplicate prevention, rollback strategy.

P24 — Onboarding

- **Status:** Planned
- **Objective:** Guide a new business to first usable accounting workflow.
- **Key Deliverables:** Organisation, financial year, currency, COA, bank, contacts; Progress tracking; First invoice.
- **Accounting:** Opening balances and finance setup must use controlled accounting flows.
- **Primary Test Gate:** Resume/skip rules, permissions, no hidden defaults.

P25 — blynt Subscriptions

- **Status:** Planned
- **Objective:** Monetise blynt with configurable SaaS plans.
- **Key Deliverables:** Stripe Billing architecture; Trials/upgrades/downgrades/cancellation; Plan capability/limit model.
- **Accounting:** Subscription billing is blynt platform commerce, separate from customer books.

- **Primary Test Gate:** Webhook idempotency, failed payment state, entitlement tests.

P26 — Public API & Webhooks

- **Status:** Planned
- **Objective:** Expose controlled integration capabilities.
- **Key Deliverables:** Hashed API keys and scopes; Rate limits and idempotency; Signed webhooks and delivery logs.
- **Accounting:** API writes still call the same domain/posting services.
- **Primary Test Gate:** Scope enforcement, replay protection, tenant boundaries.

P27 — Integrations

- **Status:** Planned
- **Objective:** Add provider-based external services without coupling them to accounting core.
- **Key Deliverables:** Stripe/PayPal/Google/Microsoft first where justified; HMRC/Open Banking later; Provider interfaces and secure credentials.
- **Accounting:** External events become validated business events; never direct ledger writes.
- **Primary Test Gate:** Provider sandbox tests, webhook signatures, graceful failure.

P28 — AWS Development Environment

- **Status:** Planning / account prepared
- **Objective:** Build low-cost blynt DEV in AWS London while retaining current hosting.
- **Key Deliverables:** Amplify, App Runner, ECR, RDS, S3, Secrets, CloudWatch; Terraform and GitHub OIDC; Sanitised/test database migration and integrity checks.
- **Accounting:** Cloud migration must preserve identical accounting integrity.
- **Primary Test Gate:** Row counts, posted debit/credit totals, trial balance and app smoke tests.

P29 — Production Hardening & Commercial Readiness

- **Status:** Planned
- **Objective:** Prepare blynt for paying businesses and controlled production cutover.
- **Key Deliverables:** MFA, CSP, monitoring, backups, incident response; Staging/production IaC and rollback runbooks; UK GDPR and security programme.
- **Accounting:** No production cutover unless financial validation passes.
- **Primary Test Gate:** Pen test readiness, backup restore drills, release gates.

P30 — blynt Intelligence

- **Status:** Planned
- **Objective:** Add explainable AI assistance only after accounting foundations are stable.
- **Key Deliverables:** Category and bank-match suggestions; Anomaly/duplicate detection; Cash-flow and report explanations.
- **Accounting:** AI never silently posts journals; financially material actions need human confirmation.

- **Primary Test Gate:** Confidence/explanation tests, permission boundaries, audit of accepted suggestions.

10. AWS Architecture & Migration

10.1 Development Target

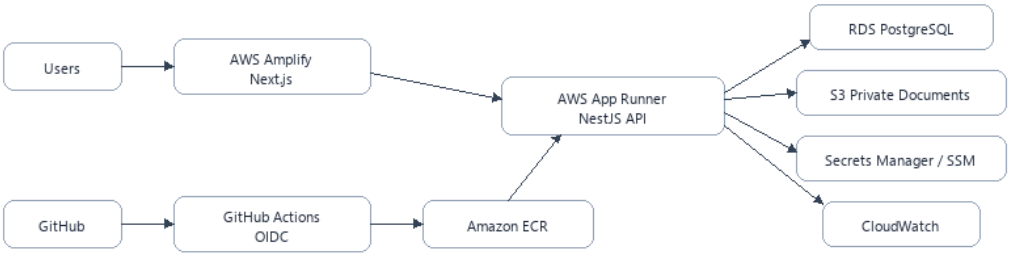


Figure 8 - Low-cost AWS DEV architecture in eu-west-2

Figure 8 - Low-cost AWS DEV architecture in eu-west-2

- **Architecture in eu-west-2:** Amplify for Next.js, App Runner for NestJS API, ECR for immutable images, Single-AZ RDS PostgreSQL, private S3, Secrets Manager/SSM and CloudWatch.
- **Cost Controls:** Expensive components like NAT Gateways, Multi-AZ, ALB/WAF, Redis or large ECS clusters are deferred until traffic warrants them.

10.2 Migration Process

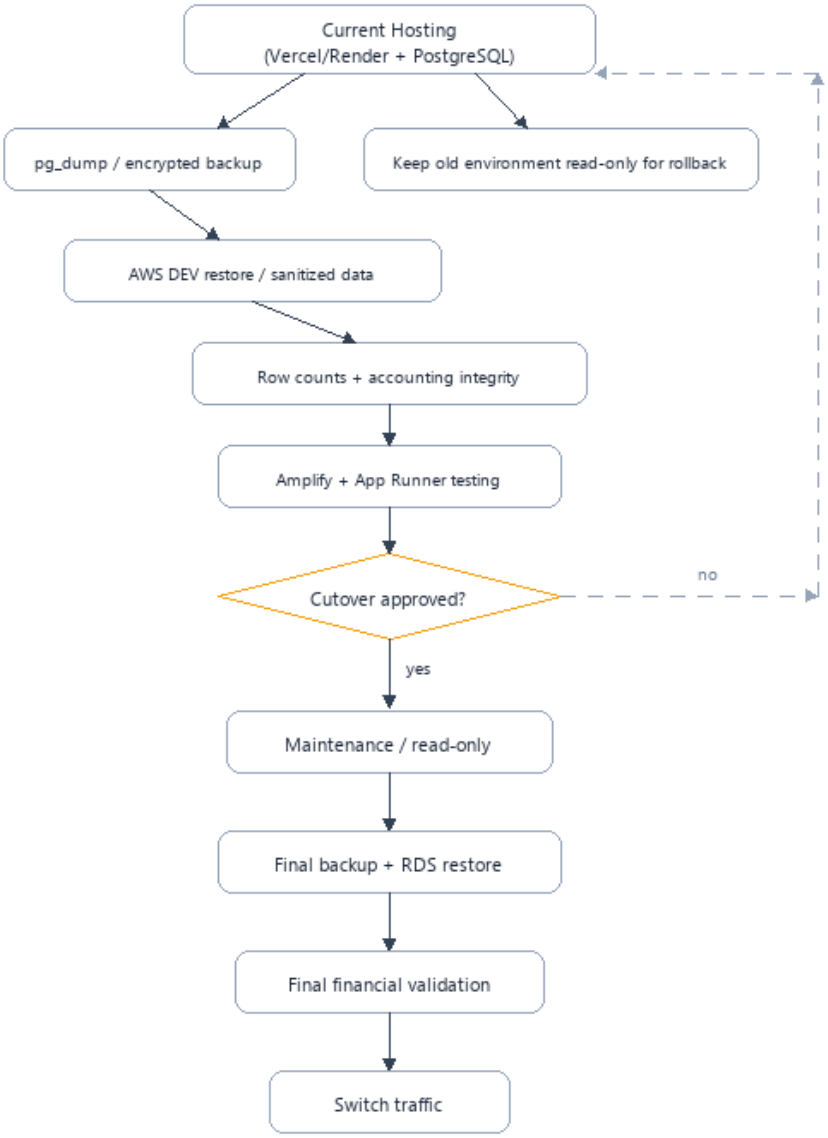


Figure 9 - Safe current-hosting to AWS migration and rollback path

- Keep current hosting online during AWS DEV validation.
- Use sanitised/test data for DEV rather than copying live personal data by default.
- For final production cutover: maintenance/read-only, final pg_dump, RDS restore, row counts, financial integrity, Trial Balance, app tests, then traffic switch.
- Keep old environment read-only for a defined rollback window.
- Never allow two production environments to accept financial writes simultaneously.

10.3 Cutover Financial Gates

The migration must NOT be approved unless all six financial integrity gates pass:

11. `Source journal count = target journal count`
12. `Source journal-line count = target journal-line count`
13. `Source posted debits = target posted debits`
14. `Source posted credits = target posted credits`
15. `Source Trial Balance totals = target Trial Balance totals`
16. `Target Trial Balance debit = target Trial Balance credit`

11. CI/CD & Release Engineering

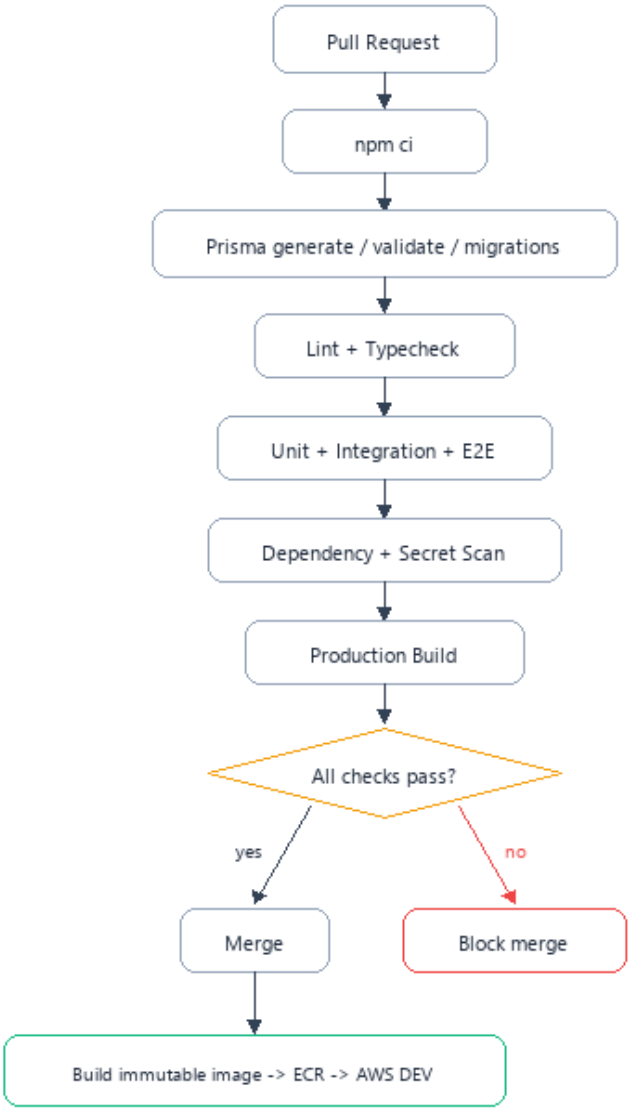


Figure 10 - Pull-request quality gate and AWS DEV deployment flow

- **Mandatory Pipeline Steps:**

17. `npm ci`
18. `Prisma generate / validate / migrations`
19. `Lint + Typecheck`
20. `Unit + Integration + E2E`
21. `Dependency + Secret Scan`

22. `Production Build`

- **Release Rules:**
- GitHub Actions OIDC for AWS deployment (zero static secrets stored in GitHub).
- Immutable ECR image tags based on commit SHA.

12. Testing Strategy

12.1 Core Accounting Regression Suite

- Debit equals credit across all posted records.
- Draft and validated journals do not hit ledger.
- Posting happens exactly once; double-click/retry cannot double-post.
- Reversal offsets original and preserves history.
- Archived-account exact reversal works while manual posting remains blocked.
- Locked periods reject posting.
- Tenant A cannot access or reference Tenant B data.
- Invoices, payments, bills and VAT produce correct journals.
- Trial Balance balances and financial statements reconcile to ledger.
- CSV exports are not truncated and neutralise spreadsheet formula injection.

12.2 Test Pyramid

- **Unit Tests:** Domain calculations, mathematical rounding, period resolution.
- **Integration Tests:** Services, database constraints, composite foreign keys.
- **E2E Tests:** Authentication, RBAC, tenant isolation, and critical business workflows.
- **Smoke Tests:** Validates deployed AWS container environments.

13. Data & Database Governance

- **PostgreSQL is authoritative:** Use Prisma migrations for controlled schema evolution.
- **Money uses NUMERIC/DECIMAL:** Managed through `MoneyService` and `Decimal.js`.
- **Use unique constraints and check constraints for domain invariants.**
- **Prefer RESTRICT rather than destructive cascades for historical accounting data.**
- **Large exports paginate or stream:** No silent 10k truncation.
- **Backups and migrations must be verifiable through row counts and accounting totals.**

13.1 Planned Schema Evolution

- Contacts: legal/contact/address/currency/VAT/company/payment-term fields and AP control relationship.
- TaxRates table with organisation-scoped codes/rates/control accounts.
- Customer payments and allocations.
- Credit notes and lines.
- Supplier bills and lines.
- Supplier payments and allocations.
- Bank accounts, bank transactions and reconciliations.

14. Compliance & Operations

14.1 UK Compliance

- UK GDPR privacy notice and processing documentation.
- Data Processing Agreement and subprocessor list.
- Retention, export and deletion workflows consistent with legal/accounting retention.
- Incident response and security communication procedures.
- Cyber Essentials $ o$ Cyber Essentials Plus $ o$ ISO 27001 readiness; SOC 2 by customer demand.

14.2 Backups & Disaster Recovery

- RDS automated backups and point-in-time recovery for production.
- Encrypted backup retention and periodic restore testing.
- S3 versioning where appropriate.
- Document and periodically review Recovery Point Objective (RPO) and Recovery Time Objective (RTO).
- Accounting integrity validation is part of recovery, not merely database availability.

15. Documentation Governance

Documentation is part of the Definition of Done. Every material code, schema, accounting, security or infrastructure change updates the relevant repository documentation and diagrams in the same phase.

- Update phase document status, objective, scope, implementation and completion report.
- Update architecture and accounting diagrams when flows change.
- Update API/database/security/AWS documents as applicable.

- Record migrations and rollback steps.
- Record tests executed and known risks.
- Never place secrets, tokens or credentials in documentation.

Appendix A - Recommended Repository Documentation Structure

```
docs/
 00-overview/
 01-architecture/
 02-accounting/
 03-modules/
 04-phases/
 05-api/
 06-database/
 07-security/
 08-aws/
 09-devops/
 10-testing/
 11-compliance/
 12-operations/
 diagrams/
```

Appendix B - Stable Domain Error Examples

- `PERIOD_LOCKED`: Attempted posting to a locked accounting period.
- `PERIOD_NOT_FOUND`: Transaction date falls outside defined financial periods.
- `ACCOUNT_NOT_FOUND`: Nominal account does not exist or belongs to another organisation.
- `INVOICE_NOT_FOUND`: Invoice ID invalid or inaccessible.
- `JOURNAL_UNBALANCED`: Debits do not equal credits down to 4 decimal places.
- `PERMISSION_DENIED`: User role lacks privileges for the requested operation.
- `TENANT_MISMATCH`: Cross-tenant reference rejected at database boundary.

Appendix C - Glossary

Term	Meaning
AR	Accounts Receivable
AP	Accounts Payable

COA	Chart of Accounts
GL	General Ledger
TB	Trial Balance
RBAC	Role-Based Access Control
MTD	Making Tax Digital
RPO	Recovery Point Objective
RTO	Recovery Time Objective
OIDC	OpenID Connect; used for GitHub-to-AWS federation
IaC	Infrastructure as Code

END OF BLYNT TECHNICAL DOCUMENTATION v1.0